Chapter 1

Hanging chain, the last episode: the Interior Point Method

Generic Inequality-Constrained QP

$$\min_{x \in \mathbb{R}^n} \quad \frac{1}{2} x^T Q x + c^T x \tag{1.1}$$

$$\text{s.t.} \quad \forall j \in \mathbb{N}_1^m \quad a_j^T x + b_j \geq 0 \tag{1.2}$$

1.1 Type of Problem

The problem is convex quadratic program with affine inequality constraints.

1.2 Convexity of Barrier function

The feasible set remains unchanged in this formulation, so convexity of the problem depends on the convexity of the objective:

$$f^{[k]}(x) = f(x) - \tau^{[k]} \sum_{j=1}^{m} \log\left(a_j^T x + b_j\right) \tag{1.3}$$

Assuming $f(x)$ is convex, then since addition preserves convexity, the function is convex if $-\tau^{[k]} \sum_{j=1}^{m} \log\left(a_j^T x + b_j\right)$ is convex. The sum preserves convexity, so does the application of a convex function such as $-\log(\cdot)$ (negative since $\tau > 0$). The argument to the logarithm is an affine function in x and therefore convex.

$\Longrightarrow$ If $f(x)$ is convex, then the problem is convex (e.g. $Q \succeq 0$)

1.3 Gradient and Hessian

$$\nabla f^{[k]}(x) = Qx + c - \tau^{[k]} \sum_{j=1}^{m} \frac{a_j}{a_j^T x + b_j} \tag{1.4}$$

$$\nabla^2 f^{[k]}(x) = Q + \tau^{[k]} \sum_{j=1}^{m} \frac{a_j a_j^T}{\left(a_j^T x + b_j\right)^2} \tag{1.5}$$

1.4 Update Rule

For a single exact Newton's step:

$$x^{[k+1]} = x^{[k]} - t\left(\nabla^2 f^{[k]}\left(x^{[k]}\right)\right)^{-1} \nabla f^{[k]}\left(x^{[k]}\right) \tag{1.6}$$

where t is found with backtracking line search on Armijo-condition

1.5 IPM Implementation

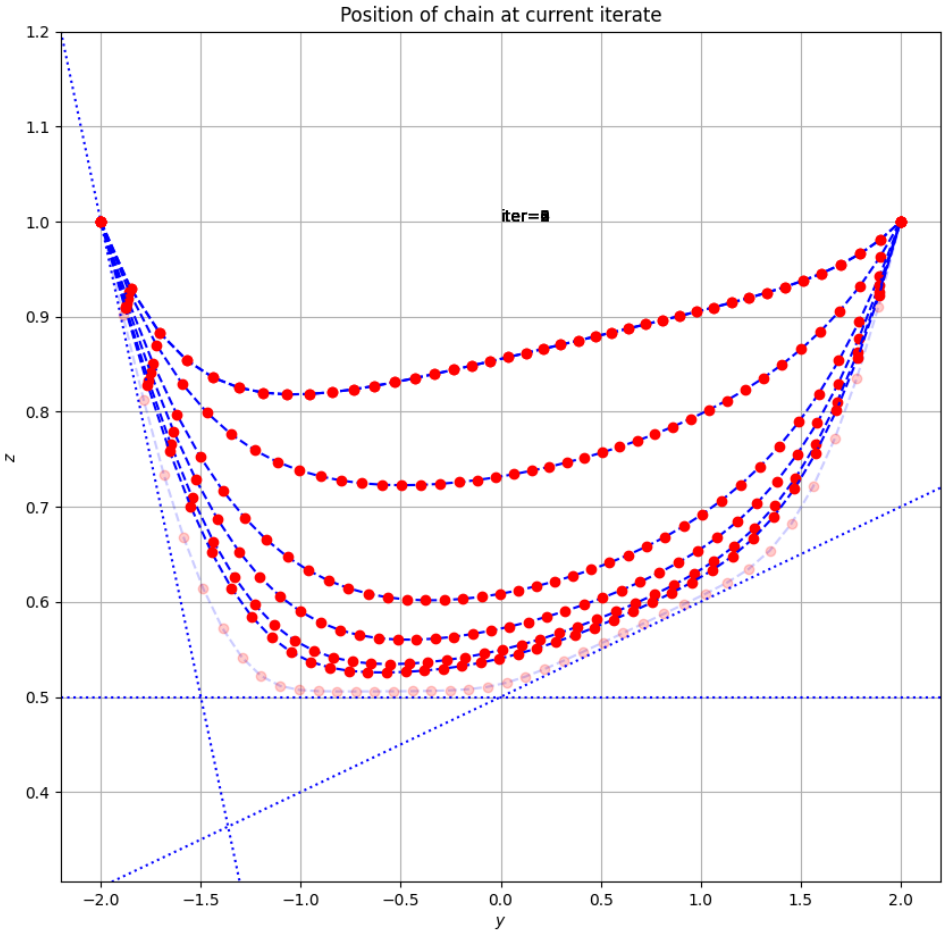

Figure 1.1: Convergence of the inequality-constrained hanging chain problem using the Interior Point Method described in this exercise in 14 iterations, illustrated by superposition of intermediate solutions. For Armijo-Backtracking, $\gamma = \frac{1}{4}, \beta = 0.8$ was chosen. The code is shown in chapter 4

Chapter 2

Sequential Quadratic Programming

Generic NLP

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) && (2.1)\\ \text{s.t.} \quad & g(x) = 0 && (2.2)\\ & h(x) \geq 0 && (2.3) \end{aligned}$$

2.1 KKT-Conditions of generic NLP

$$\begin{aligned} \nabla f(x) - \nabla h(x)\mu - \nabla g(x)\lambda &= 0 && (2.4)\\ g(x) &= 0 && (2.5)\\ h(x) &\geq 0 && (2.6)\\ \mu &\geq 0 && (2.7)\\ \forall i \in \mathbb{N}_1^q : \quad \mu_i h_i(x) &= 0 && (2.8) \end{aligned}$$

2.2 QP-subproblem at $x^{[k]}$

Let $p \coloneqq x - x^{[k]}$ and $B_k = \nabla_x^2 f\left(x^{[k]}\right)$. Linearize the constraints and approximate the objective by a quadratic form using Taylor expansion:

QP-subproblem

$$\begin{aligned} \min_{p \in \mathbb{R}^n} \quad & \nabla f(x)^T p + \frac{1}{2} p^T B_k p\\ \text{s.t.} \quad & h\left(x^{[k]}\right) + \nabla h\left(x^{[k]}\right)^T p \geq 0\\ & g\left(x^{[k]}\right) + \nabla g\left(x^{[k]}\right)^T p = 0 \end{aligned}$$

2.3 Proofs

TODO

Chapter 3

A sample exam question

Problem

$$\begin{aligned} \min_{x\in\mathbb{R}^2} \quad & x_2^4 + (x_1+2)^4 && (3.1)\\ \text{s.t.} \quad & x_1 - x_2 = 0 && (3.2)\\ & x_1^2 + x_2^2 \geq 8 && (3.3) \end{aligned}$$

3.1 Variables and Constraints

There are two scalar decision variables (x_1, x_2), one equality constraint (Equation 3.2) and one inequality constraint (Equation 3.3).

3.2 Sketch feasible set

Imagine a $45\,\mathrm{deg}$ diagonal line (horizontal axis is x_1, vertical is x_2) through the origin and a circle of radius $\sqrt{8}$ around the origin, where the parts of the line inside the circle are highlighted and labelled as Ω

3.3 Standard Form

$$f(x) = x_2^4 + (x_1+2)^4 \tag{3.4}$$
$$g(x) = x_1 - x_2 \tag{3.5}$$
$$h(x) = 8 - x_1^2 - x_2^2 \tag{3.6}$$

then:

$$\begin{aligned} \min_{x\in\mathbb{R}^2} \quad & f(x) && (3.7)\\ \text{s.t.} \quad & g(x) = 0 && (3.8)\\ & h(x) \geq 0 && (3.9) \end{aligned}$$

3.4 Convexity

The problem is convex.

$f(x)$ is convex in x, since

$$\nabla f(x) = \begin{bmatrix} 4(x_1+2)^3 \\ 4x_2^3 \end{bmatrix}$$

$$\nabla^2 f(x) = \begin{bmatrix} 12(x_1+2)^2 & 0 \\ 0 & 12x_2^2 \end{bmatrix}$$

$$\forall a,b,x_1,x_2 \in \mathbb{R}: \quad \begin{bmatrix} a & b \end{bmatrix} \begin{bmatrix} 12(x_1+2)^2 & 0 \\ 0 & 12x_2^2 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \underbrace{a^2}_{\geq 0} \cdot \underbrace{12(x_1+2)^2}_{\geq 0} + \underbrace{b^2}_{\geq 0} \cdot \underbrace{12x_2^2}_{\geq 0} \geq 0$$

Ω is convex since it is the intersection of two convex sets, namely a line defined by the affine equality constraint g and a disk defined by h (which is the level set of a convex quadratic function)

3.5 Lagrangian

$$\begin{aligned}\mathcal{L}(x,\lambda,\mu) &= f(x) - \lambda^T g(x) - \mu^T h(x) \\ &= \underbrace{x_2^4 + (x_1+2)^4}_{f(x)} \underbrace{-\lambda(x_1 - x_2)}_{-\lambda^T g(x)} \underbrace{-\mu\left(8 - x_1^2 - x_2^2\right)}_{-\mu^T h(x)}\end{aligned}$$

3.6 Active set at $\bar{x}$

At $\bar{x} = \begin{bmatrix}-2\\-2\end{bmatrix}$, the active set is $\mathcal{A} = \{1\}$ since the first and only inequality constraint is active ($h(\bar{x}) = 8 - 2^2 - 2^2 = 0$).

3.7 LICQ at $\bar{x}$

$$\nabla g(\bar{x}) = \begin{bmatrix}1\\-1\end{bmatrix} \tag{3.10}$$

$$\nabla h(\bar{x}) = \begin{bmatrix}-2x_1\\-2x_2\end{bmatrix} = \begin{bmatrix}4\\4\end{bmatrix} \tag{3.11}$$

$$\tag{3.12}$$

where $\nabla g(\bar{x}) \cdot \nabla h(\bar{x}) = 4 - 4 = 0 \implies g(\bar{x}), h(\bar{x})$ are linearly independent, so LICQ holds at $\bar{x}$

3.8 Active set at x^*

At $x^* = \begin{bmatrix}-1\\-1\end{bmatrix}$ we have $h(x^*) = 8 - 1 - 1 = 6 > 0$, so the only inequality constraint is inactive and $\mathcal{A}(x^*) = \emptyset$

3.9 LICQ at x^*

Since $\mathcal{A}(x^*) = \emptyset$, it remains to show that $\{\nabla g(x^*)\}$ is linearly independent, which is trivially true, so LICQ holds at x^*

3.10 Tangent Cone at x^*

$$T_\Omega(x^*) = \left\{\forall x \in \mathbb{R}^2 : x \cdot \begin{bmatrix}1\\-1\end{bmatrix} = 0 \,\middle|\, x\right\} \tag{3.13}$$

$$= \left\{\forall t \in \mathbb{R} \,\middle|\, t\begin{bmatrix}1\\1\end{bmatrix}\right\} \tag{3.14}$$

3.11 KKT-conditions at x^*

The KKT-conditions using $\nabla f(x), \nabla g(x), \nabla h(x)$ as derived above read:

$$\begin{bmatrix}4(x_1+2)^3\\4x_2^3\end{bmatrix}-\begin{bmatrix}-2x_1\\-2x_2\end{bmatrix}\mu-\begin{bmatrix}1\\-1\end{bmatrix}\lambda=0 \tag{3.15}$$

$$x_1-x_2=0 \tag{3.16}$$

$$8-x_1^2-x_2^2\geq 0 \tag{3.17}$$

$$\mu\geq 0 \tag{3.18}$$

$$\mu\left(8-x_1^2-x_2^2\right)=0 \tag{3.19}$$

So at $x^*=\begin{bmatrix}-1\\-1\end{bmatrix}$:

$$\begin{bmatrix}4\\-4\end{bmatrix}-\begin{bmatrix}2\\2\end{bmatrix}\mu-\begin{bmatrix}1\\-1\end{bmatrix}\lambda=0 \tag{3.20}$$

$$-1+1=0 \tag{3.21}$$

$$6\geq 0 \tag{3.22}$$

$$\mu\geq 0 \tag{3.23}$$

$$\mu\cdot 6=0 \tag{3.24}$$

It follows from Equation 3.24 that $\mu=0$, which means that:

$$\begin{bmatrix}4\\-4\end{bmatrix}-\begin{bmatrix}1\\-1\end{bmatrix}\lambda=0 \tag{3.25}$$

which implies $\lambda=4$.

All KKT-conditions are satisfied for $\mu^*=0,\lambda^*=4$

3.12 SONC at x^*

The SONC for x^* are:

1. $\exists\lambda^*,\mu^*$ such that KKT-conditions hold at x^*
2. $\forall p\in C(x^*,\mu^*):p^T\,\nabla_x^2\mathcal{L}(x^*,\lambda^*,\mu^*)p\geq 0$

The former was shown in the previous answer, with $\lambda^*=4,\mu^*=0$. The latter requires the critical cone:

$$C(x^*,\mu^*)=\left\{\forall p\in\mathbb{R}^2:\nabla f(x^*)\cdot p=0\,\middle|\,p\right\} \tag{3.26}$$

$$=\left\{\forall p\in\mathbb{R}^2:\begin{bmatrix}4\\-4\end{bmatrix}\cdot p=0\,\middle|\,p\right\} \tag{3.27}$$

$$=\left\{\forall t\in\mathbb{R}\,\middle|\,\begin{bmatrix}t\\t\end{bmatrix}\right\} \tag{3.28}$$

We also need:

$$\mathcal{L}(x,\lambda^*,\mu^*)=x_2^4+(x_1+2)^4-\lambda^*(x_1-x_2)\underbrace{-\mu^*\left(8-x_1^2-x_2^2\right)}_{=0\text{ since }\mu^*=0} \tag{3.29}$$

$$=x_2^4+(x_1+2)^4-\lambda^*x_1-\lambda^*x_2 \tag{3.30}$$

$$\nabla_x\mathcal{L}(x,\lambda^*,\mu^*)=\begin{bmatrix}4(x_1+2)^3-\lambda^*\\4x_2^3-\lambda^*\end{bmatrix} \tag{3.31}$$

$$\nabla_x^2\mathcal{L}(x,\lambda^*,\mu^*)=\begin{bmatrix}12(x_1+2)^2&0\\0&12x_2^2\end{bmatrix} \tag{3.32}$$

$$\nabla_x^2\mathcal{L}(x^*,\lambda^*,\mu^*)=\begin{bmatrix}12&0\\0&12\end{bmatrix}=12\mathbb{I} \tag{3.33}$$

to show that for all $p \in C(x^*, \mu^*)$:

$$p^T \nabla_x^2 \mathcal{L}(x^*, \lambda^*, \mu^*) p = \begin{bmatrix} t & t \end{bmatrix} (12\mathbb{I}) \begin{bmatrix} t \\ t \end{bmatrix} \tag{3.34}$$

$$= \begin{bmatrix} t & t \end{bmatrix} \begin{bmatrix} 12t \\ 12t \end{bmatrix} \tag{3.35}$$

$$= 12t^2 + 12t^2 \tag{3.36}$$

$$= 24t^2 \geq 0 \tag{3.37}$$

so x^* fulfills SONC.

Also, since $C(x^*, \mu^*) \backslash \{\vec{0}\} = \left\{ t \in \mathbb{R} : t \neq 0 \;\middle|\; \begin{bmatrix} t \\ t \end{bmatrix} \right\}$ and $t \neq 0 \implies 24t^2 > 0$ and SONC hold, it follows that SOSC hold, so x^* is a strict local minimizer.

Since x^* is a *strict* local minimizer of a convex problem, this actually implies that x^* is the global minimum.

Chapter 4

Code

```
"""
    Leo Simpson, University of Freiburg (teacher assistant), 2025.

    This file is for an exercise for the course Numerical Optimization by Prof. Moritz Diehl.
"""

import numpy as np
import matplotlib.pyplot as plt

from hanging_chain_ip_matrices import create_matrices, N
from hanging_chain_ip_animation import make_animation


Q, c, A, b = create_matrices() # Create the matrices for the problem
M = b.shape[0]

def f(x, tau):
    # Objective function for a given tau
    s = A @ x + b # the constraint is s > 0
    if np.any(s <= 0):
        return np.inf
    else:
        # Objective function for the subproblem with log barrier
        return 0.5* np.dot(x, (Q @ x)) + np.dot(c, x) - tau*np.sum(np.log(s))

y_list, z_list = [], []
max_iter = 1000
x_k = np.ones(2*(N-1))
tol = 1e-1

for i in range(max_iter):
    print(f"Iteration {i+1}")
    tau_k = 1 / (i+1)**2

    # Compute the constraint residual
    s_k = A @ x_k + b # the constraint is s_k > 0
    assert np.all(s_k > 0), "Constraint violated !"

    # Compute the gradient
    grad_f = Q @ x_k + c
    grad = grad_f - tau_k * np.sum([1/s_k[j]*A[j].T for j in range(M)], axis=0)

    # Check convergence
    inf_norm_grad = np.max(abs(grad))
    print(f"Iteration {i+1}, grad = {inf_norm_grad:.2e}")
    if inf_norm_grad < tol:
        print("Converged !")
        break
```

```
    # Compute the Hessian
    H_k = Q + tau_k * np.sum([1/s_k[j]**2 * np.outer(A[j], A[j]) for j in range(M)], axis=0)

    # Newton step
    dx = -(np.linalg.inv(H_k)) @ grad_f

    # Globalization with backtracking line-search and Armijo criterion
    t = 1         # t_max
    gamma = 0.25  # gamma in (0, 1/2)
    beta = 0.8    # beta  in (0,  1 )
    while f(x_k + t * dx, tau_k) >= f(x_k, tau_k) + gamma*t*np.dot(grad, dx):
        t *= beta
    print(f"Armijo step length: t={t}")

   # Update the solution
    x_k = x_k + t * dx

    # Save variables
    y_list.append(x_k[:N-1])
    z_list.append(x_k[N-1:])

anim = make_animation(y_list, z_list)
plt.show()
```